

Security Audit

Scall (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	30
Conclusion	31
Our Methodology	32
Disclaimers	34
About Hashlock	35

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Scall team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Scall.io is a decentralized finance (DeFi) protocol focused on perpetual options trading built on Web3 infrastructure. It enables users to trade call and put options on assets like BTC and ETH with no expiration dates, pay fees per second, and potentially earn unlimited profit while limiting risk to the cost of holding positions. The platform is non-custodial, meaning traders retain control of their crypto wallets, and integrates features such as liquidity provision with automatic APR earnings for LPs, instant execution, and a fully decentralized smart-contract base that removes traditional intermediaries. Scall.io emphasizes security, decentralization, and seamless on-chain trading, allowing participants to open, manage, and close perpetual options positions directly from their connected wallets.

Project Name: Scall

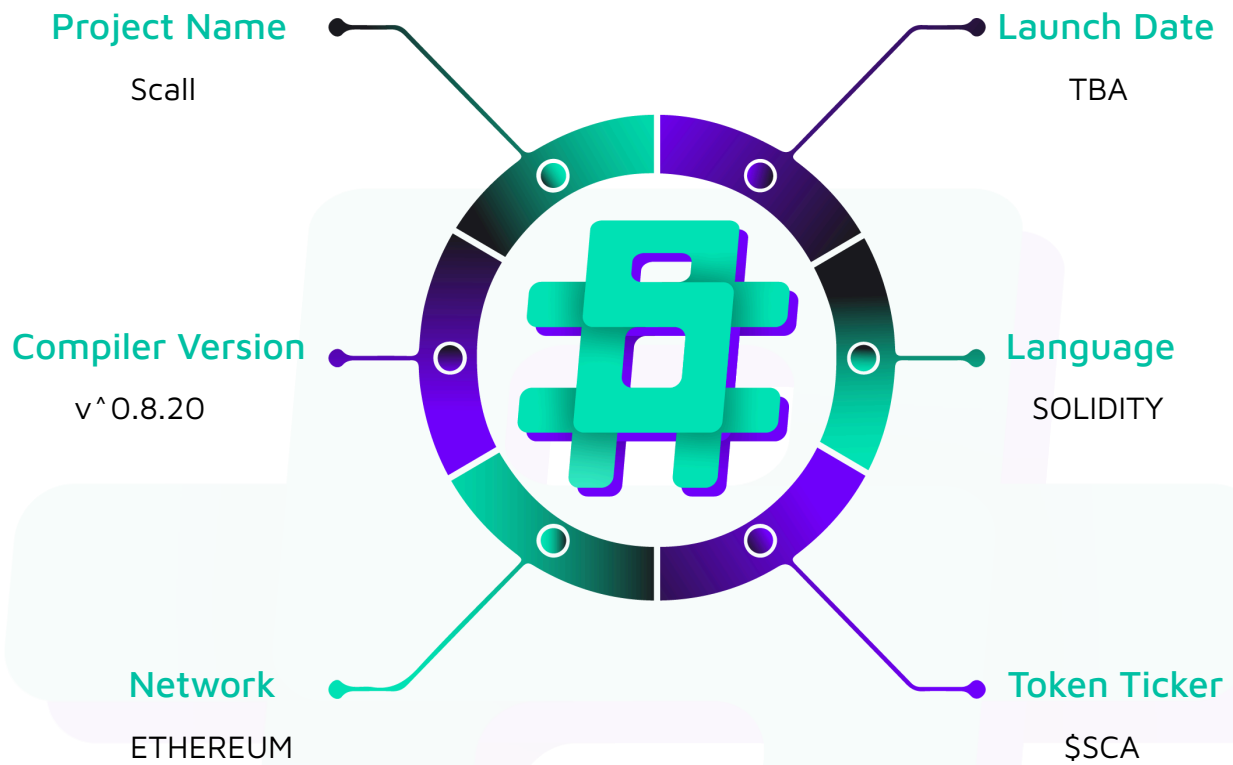
Project Type: DeFi

Compiler Version: ^0.8.20

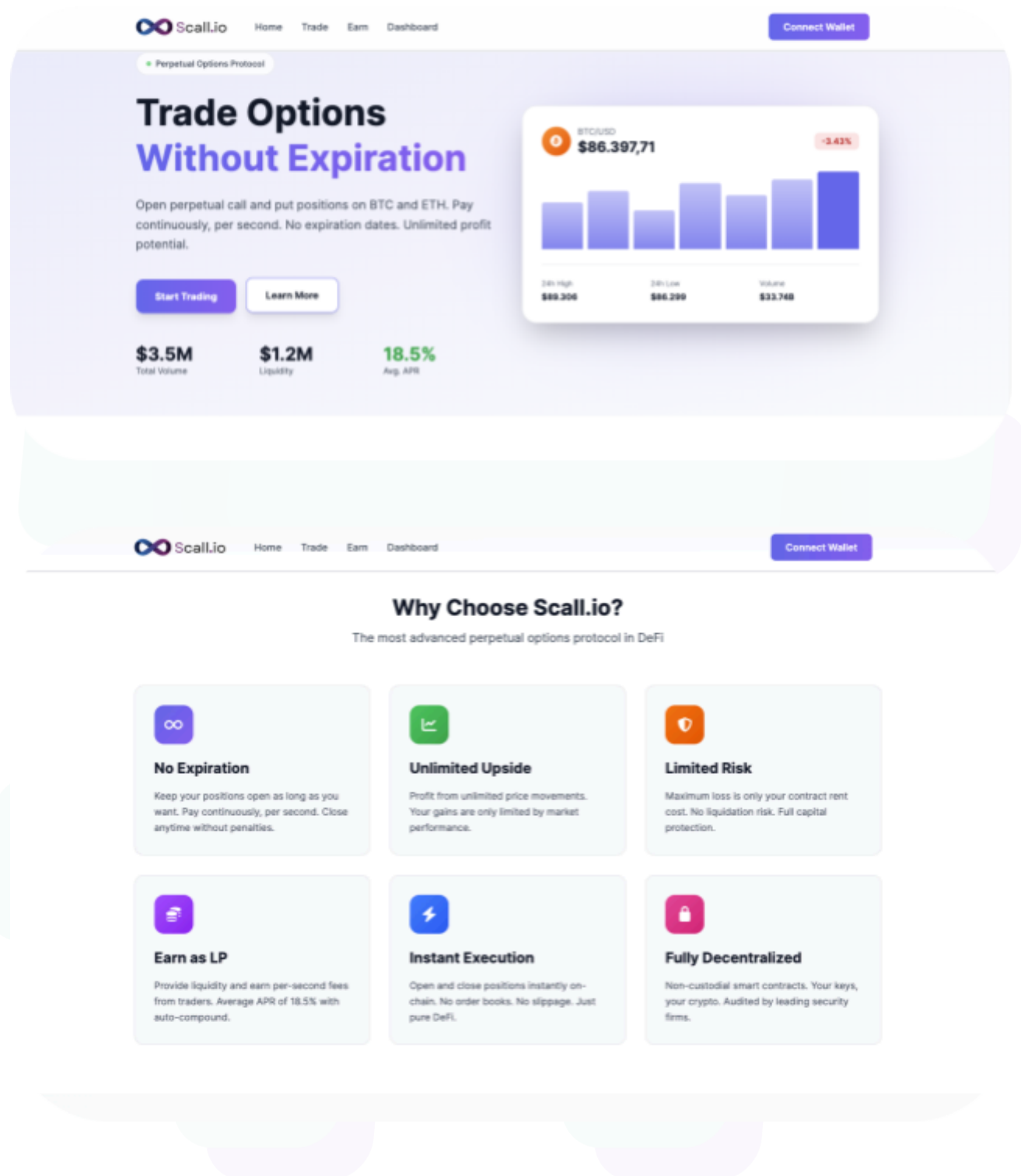
Website: <https://scall.io/en/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Scall project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Scall Smart Contracts
Platform	Ethereum / Solidity
Audit Date	January, 2026
Contract 1	Main.sol
Contract 2	CollateralPool.sol
Contract 3	MarketPool.sol
Contract 4	ERC721_LP.sol
Contract 5	ERC721_Contract.sol
Audited GitHub Commit Hash	2d32732fc4436eb1f790cbdc9217ef71a08875ba
Fix Review GitHub Commit Hash	e9a1def5c22c4c7c05e51b8076b5917efe312327

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

4 Medium severity vulnerabilities

3 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
MarketPool.sol - Allows the owner to: - Set price feed address and decimals - Set range parameter - Set interval length - Allows liquidity providers (LPs) to: - Deposit liquidity for call options (Token A) - Deposit liquidity for put options (Token B) - Withdraw liquidity and receive proportional assets - Claim rewards through CollateralPool - Allows option traders to: - Open call option contracts at available strikes - Open put option contracts at available strikes - Close call option contracts (settle ITM or OTM) - Close put option contracts (settle ITM or OTM) - Allows the CollateralPool to: - Liquidate undercollateralized option contracts - Claim rewards on behalf of LP owners - Allows anyone to: - Query LP position information - Query contract position information - Query strike information and history - Query available strike intervals - Query the current price from the oracle - Calculate rent for potential positions - Calculate rewards for LP positions	Contract achieves this functionality.
CollateralPool.sol - Allows users to:	Contract achieves this functionality.

- Deposit collateral tokens - Withdraw collateral tokens (if sufficient balance) - Claim LP rewards from any market - View their collateral balance - View their accumulated fees - Allows liquidators to: - Liquidate undercollateralized positions - Receive liquidation penalty as reward - Allows MarketPools to: - Update user rent and collateral information - Query if the user can open new contracts - Allows anyone to: - Query user information (collateral, rent, lastUpdate) - Query if a user needs liquidation - Calculate rewards for LP positions (view)	
Main.sol - Allows the owner to: - Set collateral pool address - Set liquidation threshold - Set liquidation penalty - Set minimum collateral requirement - Set protocol fees - Link new markets to the protocol - Update existing market parameters (price feed, range, interval) - Allows anyone to: - Query collateral token address - Query collateral token decimals - Query collateral pool address - Query liquidation threshold - Query liquidation penalty - Query minimum collateral requirement	Contract achieves this functionality.

- Query protocol fees - Query market count - Query market ID by address - Query market address by ID - Query market information by ID	
ERC721_LP.sol - Allows the MarketPool to: - Mint new LP position NFTs - Burn LP position NFTs on withdrawal - Allows NFT holders to: - Transfer LP position NFTs - Approve transfers to other addresses - Allows anyone to: - Query token ownership - Query all token IDs - Query all unique owners - Query total supply	Contract achieves this functionality.
ERC721_Contract.sol - Allows the MarketPool to: - Mint new option contract NFTs - Burn option contract NFTs on close/liquidation - Restricts: - All transfers (option contracts are non-transferable) - Allows anyone to: - Query token ownership - Query all token IDs - Query all unique owners - Query total supply	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Scall project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Scall project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] MarketPool#withdraw - Unfair Withdrawal Mechanism (Bad Asset Swap)

Description

The withdrawal mechanism in MarketPool allocates exercised proceeds (callLR/putLR) to the first withdrawing LP rather than distributing the pool's current asset mix pro rata across all LP shares. This breaks the pro-rata invariant and enables race-condition value extraction.

Vulnerability Details

When options are exercised, the returned liquidity (callLR/putLR) accumulates in the pool. The withdrawal logic gives the ENTIRE accumulated callLR/putLR to whichever LP withdraws first, rather than distributing proportionally based on LP share.

```
// Call branch - First withdrawer gets ALL callLR
availableFunds = strikeInfos.callLP - strikeInfos.callLU;

if (strikeInfos.callLR > 0) {
    liquidityReturned = (strikeInfos.callLR * 1e18)/thisLP.strike;

    if (availableFunds < thisLP.amount) {
        tokenBtoTransfer = strikeInfos.callLR; // ALL callLR goes to first withdrawer
        tokenAtoTransfer = availableFunds - liquidityReturned;

        _strikeToInfos[thisLP.strike].callLR = 0; // Zeroed out, nothing left for others
        // ...
    }
}
```

Proof of Concept

An example of a test that simulates unfair distribution as `User1` (the first withdrawer) receives Token B from the exercise, while `User2` gets pure Token A.

```
function test_UnfairWithdrawal_Scenario() public {

    // 1. Setup

    uint256 strikeIndex = 2;

    address user1 = address(0x101);

    address user2 = address(0x102);

    address opener = address(0x103);

    tokenA.mint(user1, 1000e18);

    tokenA.mint(user2, 1000e18);

    tokenB.mint(opener, 200000e18);

    // Approve

    vm.prank(user1);

    tokenA.approve(address(marketPool), 1000e18);

    vm.prank(user2);

    tokenA.approve(address(marketPool), 1000e18);

    vm.startPrank(opener);

    tokenB.approve(address(marketPool), type(uint256).max);

    tokenB.approve(address(collateralPool), type(uint256).max);

    vm.stopPrank();

    // 2. Deposits (Equal time, equal amount)

    vm.prank(user1);

    uint256 id1 = marketPool.deposit(true, strikeIndex, 100e18);

    vm.prank(user2);

    uint256 id2 = marketPool.deposit(true, strikeIndex, 100e18);
```

```

// 3. Open Contract and deposit collateral

vm.prank(opener);

collateralPool.depositCollateral(2000e18);

vm.prank(opener);

marketPool.openContract(true, strikeIndex, 50e18);

// 4. Move Price to ITM and exercise

uint256[] memory intervals = marketPool.getIntervals();

uint256 strike = intervals[5 + strikeIndex];

priceFeed.setPrice(int256(strike + 500e18));

vm.prank(opener);

marketPool.closeContract(0);

// 5. User 1 Withdraws FIRST

vm.prank(user1);

(uint256 u1_A, uint256 u1_B,) = marketPool.withdraw(id1);

// 6. User 2 Withdraws SECOND

vm.prank(user2);

(uint256 u2_A, uint256 u2_B,) = marketPool.withdraw(id2);

// PROOF OF UNFAIRNESS:

assertGt(u1_B, 0, "User 1 should receive Token B (Exercised portion)");

assertLt(u1_A, 100e18, "User 1 should receive less than full Token A");

assertEq(u2_B, 0, "User 2 should receive 0 Token B (Avoided swap)");

assertEq(u2_A, 100e18, "User 2 should receive full Token A (Protected)");

}

```

Impact

Strategic exits can extract value from passive LPs, producing systematic unfairness and potential fund loss during volatile periods.



Recommendation

It is recommended to adopt pro-rata share accounting (ERC-4626 style). On withdrawal, burn shares and transfer the pool's current asset mix proportionally based on the LP's share of total liquidity.

Status

Resolved

[H-02] MarketPool#getRewards - Permanent DoS via Unbounded Strike History Iteration

Description

Reward computation iterates over `_strikeHistory` for each claim. Since history grows with every deposit/withdraw/contract update, an attacker can intentionally bloat history to reach prohibitive gas costs, causing a claim DoS.

Vulnerability Details

The `getRewards()` function iterates backward through the entire strike history. Each deposit, withdraw, or contract operation appends to history. Gas cost grows $O(n)$ with history size.

```
// Unbounded loop - gas grows linearly with history size
for(uint256 i = ctx.requestedCount - 1; i >= 0; i--) {
    ctx.strikeCount = _strikeHistory[ctx.lp.strike][i];

    // ... reward calculation ...

    ctx.rewards += (ctx.userShare * ctx.rewardsPerStrike) / 1e18;

    if(ctx.lp.lastClaim >= ctx.strikeCount.updated) { break; }
}
```

Proof of Concept

The following test case demonstrates an attacker triggered DoS as Gas usage grows linearly with history size - 300 deposits cause a 10x+ gas increase.

```
function test_DoS_GasLimit() public {

    // 1. Setup

    uint256 strikeIndex = 2;

    address user = address(0x201);

    address spammer = address(0x202);
```

```

tokenA.mint(user, 1000e18);

tokenA.mint(spammer, 1000000e18);


vm.prank(user);

tokenA.approve(address(marketPool), 1000e18);

vm.startPrank(spammer);

tokenA.approve(address(marketPool), type(uint256).max);

vm.stopPrank();


// 2. User Deposits

vm.prank(user);

uint256 userId = marketPool.deposit(true, strikeIndex, 100e18);


// 3. Measure initial gas

uint256 startGas = gasleft();

vm.prank(user);

marketPool.getRewards(userId, 0);

uint256 gasUsedInitial = startGas - gasleft();

console.log("Gas Used Initial:", gasUsedInitial);


// 4. Spammer bloats the history with 300 deposits

uint256 iterations = 300;

vm.startPrank(spammer);

for (uint256 i = 0; i < iterations; i++) {

    vm.warp(block.timestamp + 10 minutes);

    marketPool.deposit(true, strikeIndex, 1e18);

```

```

    }

    vm.stopPrank();

    // 5. Measure gas after bloat

    vm.warp(block.timestamp + 10 minutes);

    startGas = gasleft();

    vm.prank(user);

    marketPool.getRewards(userId, 0);

    uint256 gasUsedFinal = startGas - gasleft();

    console.log("Gas Used Final:", gasUsedFinal);

    // Assert Gas Usage Increased Significantly (Linear Growth)

    assertGt(gasUsedFinal, gasUsedInitial * 10, "Gas usage should increase linearly");
}

```

Impact

Rewards become unclaimable or prohibitively expensive, resulting in economic denial of service. Attack cost is low (many small deposits), but the impact is permanent for affected strikes.

Recommendation

It is recommended to replace looped reward calculation with $O(1)$ accumulators and snapshot indexes per LP. Use a cumulative reward-per-share pattern similar to MasterChef or Synthetix staking.

Status

Resolved

[H-03] MarketPool#getPrice() - Interval Underflow Freezes Deposits and Open Contracts When Price < Range

Description

Interval computation subtracts `_RANGE` from `lowerBound` repeatedly. If `currentPrice < _RANGE`, the subtraction underflows (reverts in Solidity 0.8+), freezing deposits and contract opening.

Vulnerability Details

The interval calculation assumes price is always greater than `_RANGE * halfLength`. When price drops below `_RANGE`, the subtraction underflows.

```
uint256 modulo = currentPrice % _RANGE;

uint256 lowerBound = currentPrice - modulo;

// This underflows when lowerBound < _RANGE * (i + 1)
for (uint256 i = 0; i < halfLength; i++) {
    intervals[i] = lowerBound - _RANGE * (i + 1); // UNDERFLOW!
}
```

Proof of Concept

The following test cases demonstrate that both `deposit` and `openContract` revert when `price < range` due to underflow.

```
function test_DoS_Intervals_Underflow_Deposit() public {
    priceFeed.setPrice(50e8); // Price below _RANGE (100e18)
    vm.expectRevert();        // Expects underflow revert
    marketPool.deposit(true, 1, 1e18);
}

function test_DoS_Intervals_Underflow_OpenContract() public {
```

```
priceFeed.setPrice(50e8); // Price below _RANGE (100e18)

vm.expectRevert();        // Expects underflow revert

marketPool.openContract(true, 1, 1e18);

}
```

Impact

Market availability is lost during extreme price drops. Users cannot add liquidity or open positions until the price recovers above `_RANGE`.

Recommendation

It is recommended to clamp lower interval bounds to zero.

Status

Resolved

Medium

[M-01] MarketPool#liquidateContract - Liquidation Failure due to tx.origin Usage

Description

Liquidation pulls funds from `tx.origin` instead of `msg.sender`. Keeper contracts that hold funds will fail to liquidate unless the controlling EOA is separately funded.

Vulnerability Details

The liquidation function uses `tx.origin` for token transfers instead of `msg.sender`. This breaks compatibility with keeper contracts and smart contract wallets.

```
// Uses tx.origin instead of msg.sender  
IERC20x(_TOKENB).transferFrom(tx.origin, address(this), ...);  
IERC20x(_TOKENA).transfer(tx.origin, ...);
```

Impact

Liquidations become unreliable during volatility. Automated keeper infrastructure cannot function, increasing insolvency risk and potentially leaving underwater positions unliquidated.

Recommendation

It is recommended to use `msg.sender` instead of `tx.origin`. Require the liquidator (`msg.sender`) to have an allowance to MarketPool for the needed tokens, enabling contract-based liquidations.

Status

Resolved

[M-02] MarketPool#getPrice - Oracle Staleness Enables Mispriced Option Entry

Description

The protocol uses Chainlink spot prices without validating round freshness or staleness, allowing option entry at stale strikes and profit capture after oracle update.

Vulnerability Details

The `getPrice()` function fetches the latest price without checking if it's stale or if the oracle is functioning correctly.

```
// No staleness check - price could be hours old  
  
(, int result,,, ) = IChainlink(_PRICEFEED).latestRoundData();  
  
return uint256(result) * (1e18/10**_PRICEFEED_DECIMAL);
```

Impact

Repeatable value leakage from LPs during oracle lag intervals. Especially dangerous during high volatility or L2 sequencer downtime.

Recommendation

It is recommended to validate freshness using `updatedAt` and `answeredInRound`. Enforce maximum staleness threshold (e.g., 1 hour). Consider TWAP or multi-source aggregation for critical price points.

Status

Resolved

[M-03] MarketPool#deposit - Fee-On-Transfer / Rebasing Tokens Break LP and Collateral Accounting

Description

Deposits and collateral credit assume `_amount` equals the received amount. With fee-on-transfer or rebasing tokens, credited balances exceed actual balances, enabling over-withdrawals.

Vulnerability Details

The contracts credit the user-supplied `_amount` without checking the actual tokens received.

```
// MarketPool.deposit() - credits _amount, not actual received
IERC20x(_TOKENA).transferFrom(msg.sender, address(this), _amount);

// ... later credits _amount to LP position

// CollateralPool.depositCollateral() - same issue
IERC20x(_COLLATERALTOKEN).transferFrom(msg.sender, address(this), _amount);

_userInfos[msg.sender].collateral += collateralTokenTo18(_amount); // Credits full
amount
```

Impact

Pool solvency is threatened. Attackers can withdraw more than was received, draining funds from other LPs/users.

Recommendation

It is recommended to add a credit-based system based on observed balance deltas.

Status

Resolved

Low

[L-01] MarketPool#openContract - Rent Truncation to Zero for Micro Positions

Description

Sequential division operations lose precision. For very small amounts, the final rent rounds to zero.

Vulnerability Details

```
// Line 607 - Call rent calculation
rent = (((amount * strike) / 1e18) * _YIELD) / 1e18 / 31536000;

// Line 613 - Put rent calculation
rent = ((amount * _YIELD) / 1e18) / 31536000;
```

Impact

Positions with extremely small amounts pay zero rent, getting free option exposure. Economic impact is negligible since the amounts are too small to be meaningful.

Recommendation

We recommend to add minimum amount check or reorder operations for better precision

Status

Resolved

[L-02] MarketPool#tokenTo18 - Double Precision Loss in Decimal Conversions

Description

Token amounts undergo round-trip decimal conversions (native → 18 decimals → native). Each conversion loses precision, particularly for tokens with fewer decimals, like USDC (6 decimals).

Vulnerability Details

```
// tokenATo18: native → 18 decimals
return (_amount * 1e18) / 10 ** _TOKENA_DECIMALS;

// toTokenADecimal: 18 decimals → native
return (_amount * 10 ** _TOKENA_DECIMALS) / 1e18;
```

Impact

Dust amounts lost on each conversion. Over 1 million operations, approximately 1 token is lost. Economic impact is minimal but creates accounting discrepancies.

Recommendation

For critical precision, we recommend considering using higher internal precision (e.g., 27 decimals) or tracking remainders. Document acceptable precision loss thresholds.

Status

Acknowledged

[L-03] CollateralPool#liquidateContract - CEI Pattern Violation in liquidateContract

Description

The liquidateContract function performs external calls before updating the user's collateral state, violating the Checks-Effects-Interactions (CEI) pattern.

Vulnerability Details

```
function liquidateContract(uint256 _index, uint256 _id) external returns (uint256) {  
    // ... checks ...  
  
    uint256 penalty = (balanceOf(user) * liqPen) / 1e18;  
  
    // EXTERNAL CALL 1: MarketPool.liquidateContract  
  
    IMarketPool(IMain(_MAIN).getIdToMarket(_index)).liquidateContract(_id);  
  
    // EXTERNAL CALL 2: Transfer penalty to liquidator  
  
    IERC20x(_COLLATERALTOKEN).transfer(msg.sender, toCollateralTokenDecimals(penalty));  
  
    // STATE UPDATE LAST  
  
    _userInfos[user].collateral -= penalty;  
}
```

Impact

Violates best practice for smart contract development. State updates should precede external calls to maintain a consistent state throughout execution.

Recommendation

It is recommended follow the CEI pattern and update the state before external calls.

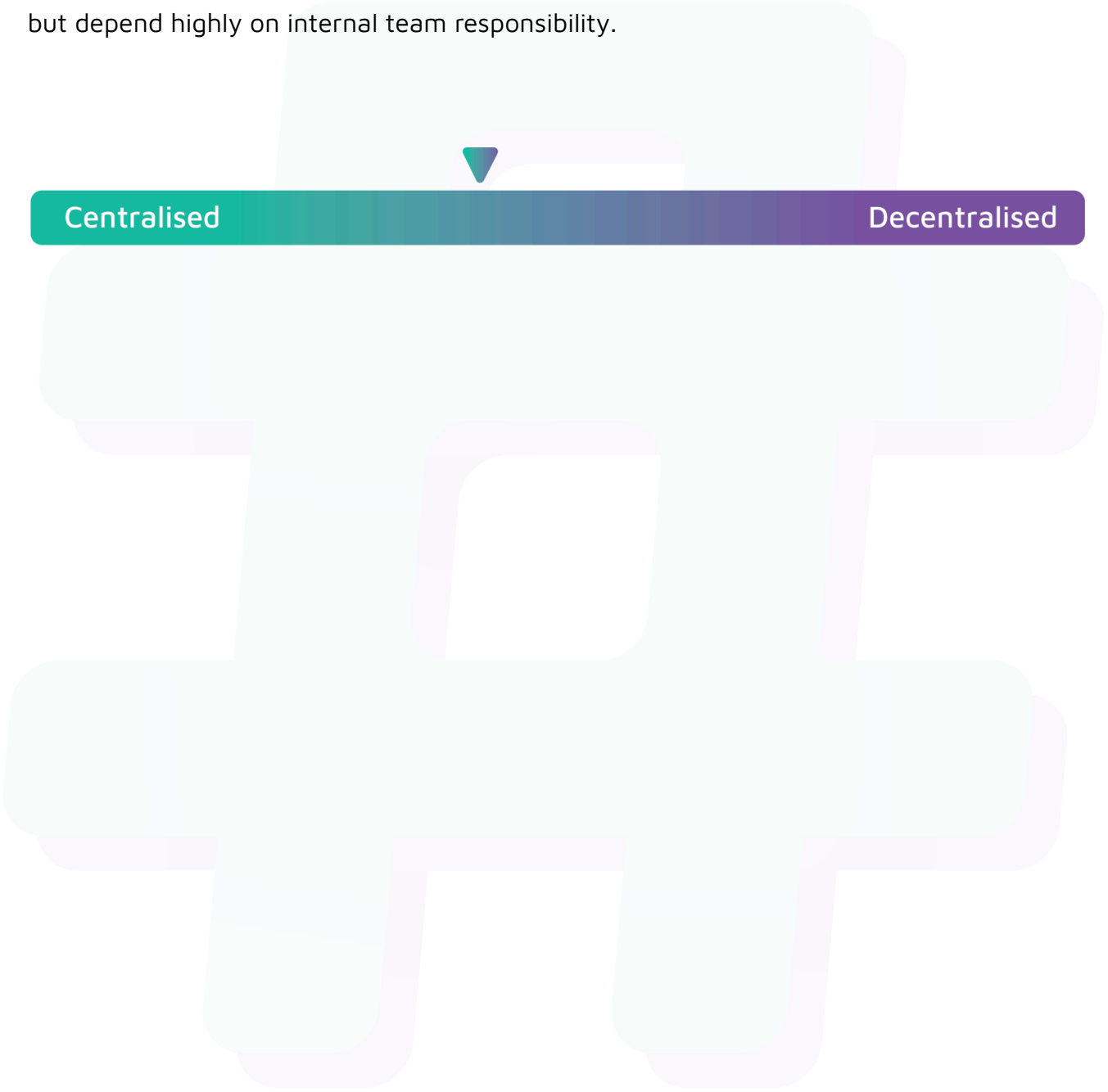
Status

Resolved

Centralisation

The Scall project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Scall project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.